Class: TE **Subject: AISC** **Sem-V**

Experiment No: 08

Course Outcome: Implement EBPTA
Blooms Level: L3 – Apply

Aim: Student Performance Prediction using EBPTA

A college wants to predict whether a student will **Pass or Fail** based on:

- **Hours Studied**
- **Attendance**

Use a small multilayer neural network trained using the **Error Back-Propagation Training Algorithm (EBPTA)**.

Training Dataset

Hours Studied	Attendance	Target
2	50	0 (Fail)
3	55	0 (Fail)
4	60	0 (Fail)
5	70	1 (Pass)
6	75	1 (Pass)
7	85	1 (Pass)

Abstract:
This experiment implements the Error Back Propagation Training Algorithm (EBPTA) using Python to train a simple multilayer neural network. The network learns to predict whether a student will Pass or Fail based on hours studied and attendance. During training, the network calculates the error between the actual and predicted output, propagates the error backward, and updates the weights and biases to improve its prediction. After training, the user can enter new student values and verify the predicted result against the expected target.

Sample Input and Output:

```
Enter Hours Studied: 6
Enter Attendance (%): 80
Enter Expected Target (0 = Fail, 1 = Pass): 1

Training Completed!

Checking Training Data:

Hours: 2 Attendance: 50 Actual: Fail Predicted: Fail
Hours: 3 Attendance: 55 Actual: Fail Predicted: Fail
Hours: 4 Attendance: 60 Actual: Fail Predicted: Fail
```

```
Hours: 5 Attendance: 70 Actual: Pass Predicted: Pass
Hours: 6 Attendance: 75 Actual: Pass Predicted: Pass
Hours: 7 Attendance: 85 Actual: Pass Predicted: Pass

---------------------------------
TEST THE TRAINED EBPTA SYSTEM
---------------------------------

EBPTA Prediction: Pass
Output Value: 0.94

To check whether the prediction is correct:

Prediction is CORRECT
```

Theory:

Error Back Propagation Training Algorithm (EBPTA) is a supervised learning algorithm used to train a multilayer neural network. It works by comparing the **actual output** with the **predicted output** and calculating the error.

The error is then **propagated backward** through the network, and the weights and biases are adjusted to reduce the error. This process is repeated for several iterations until the network learns to give better predictions.

The main steps of EBPTA are:

1. Give the input to the neural network.
2. Calculate the output using the forward pass.
3. Find the error between actual and predicted output.
4. Propagate the error backward.
5. Update the weights and biases.
6. Repeat the process for several epochs.

In this experiment, EBPTA is used to predict whether a student will **Pass or Fail** based on **hours studied and attendance**.

1. Forward Pass

The input values are passed from the input layer to the hidden layer and then to the output layer.

$$Hidden = f(XW_1 + B_1)$$

$$Output = f(HiddenW_2 + B_2)$$

where f is the sigmoid activation function.

2. Calculate Error

The predicted output is compared with the actual target.

$$Error = Target - Output$$

A smaller error indicates that the network's prediction is closer to the expected result.

3. Back Propagation

The error is propagated backward from the output layer towards the hidden layer. The algorithm calculates correction values, called **deltas**, for the neurons.

For the sigmoid function:

$$f'(x) = f(x)(1 - f(x))$$

This derivative is used to determine the amount of correction required.

4. Update Weights and Biases

The weights and biases are adjusted according to the calculated error and learning rate.

$$W_{new} = W_{old} + \eta \Delta W$$

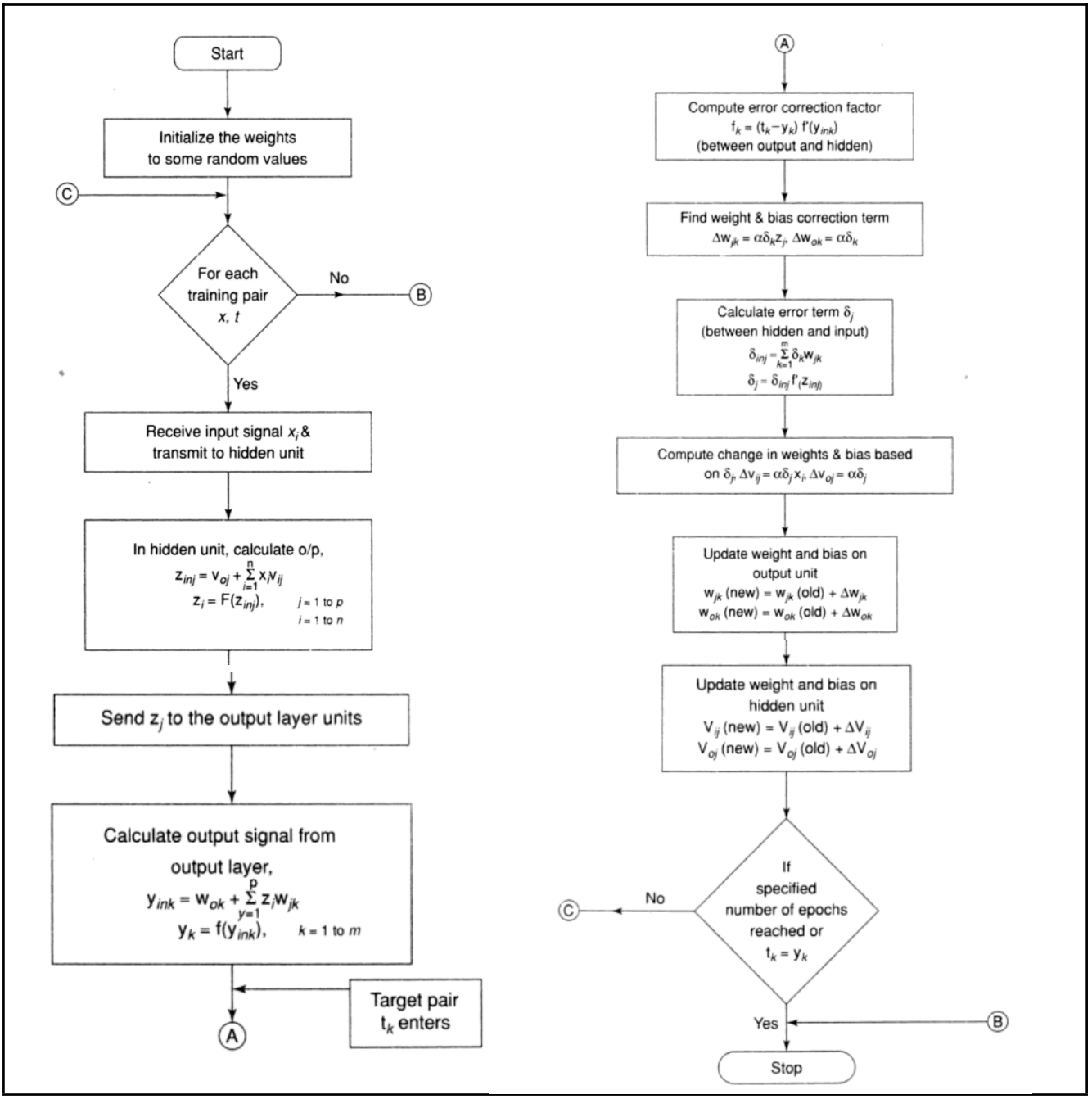
where:

- W = weight
- η = learning rate
- ΔW = weight correction

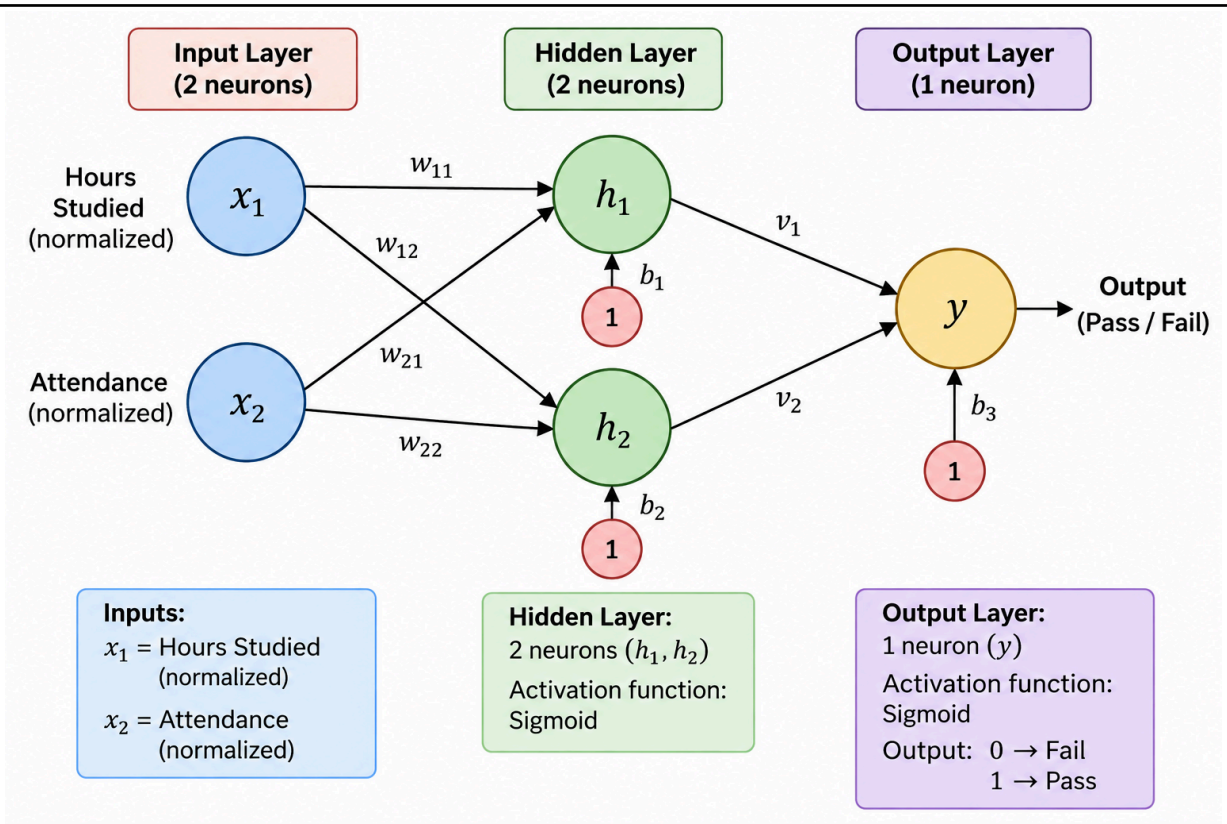
5. Repeat Training

The forward pass, error calculation, back propagation, and weight updating are repeated for a specified number of **epochs**.

Class: TE **Subject: AISC** **Sem-V**



Class: TE **Subject: AISC** **Sem-V**



Program:

```
import numpy as np

# ---------------------------------------
# 1. Training Data
# ---------------------------------------

# Inputs: [Hours Studied, Attendance]
X = np.array([
    [2, 50],
    [3, 55],
    [4, 60],
    [5, 70],
    [6, 75],
    [7, 85]
], dtype=float)
```

Class: TE **Subject: AISC** **Sem-V**

```
# Target: 0 = Fail, 1 = Pass
Y = np.array([
    [0],
    [0],
    [0],
    [1],
    [1],
    [1]
], dtype=float)
# Normalize input values
X[:, 0] = X[:, 0] / 10      # Hours
X[:, 1] = X[:, 1] / 100     # Attendance
# ----------------------------------------
# 2. Sigmoid Activation Function
# ----------------------------------------

def sigmoid(x):
    return 1 / (1 + np.exp(-x))



# ----------------------------------------
# 3. Initial Weights and Biases
# ----------------------------------------

W1 = np.array([
    [0.2, 0.3],
    [0.4, 0.5]
])

W2 = np.array([
    [0.6],
    [0.7]
])

B1 = np.array([[0.1, 0.1]])
B2 = np.array([[0.1]])
```

```
learning_rate = 0.5
epochs = 5000      #number of iteration


# ---------------------------------------
# 4. Training using EBPTA
# ---------------------------------------

for epoch in range(epochs):

    # Forward Pass
    hidden_input = np.dot(X, W1) + B1
    hidden_output = sigmoid(hidden_input)

    output_input = np.dot(hidden_output, W2) + B2
    output = sigmoid(output_input)

    # Calculate Error
    error = Y - output

    # Backward Pass
    output_delta = error * output * (1 - output)

    hidden_delta = (
        hidden_output *
        (1 - hidden_output) *
        np.dot(output_delta, W2.T)
    )

    # Update weights
    W2 = W2 + learning_rate * np.dot(hidden_output.T, output_delta)
    W1 = W1 + learning_rate * np.dot(X.T, hidden_delta)

    # Update biases
    B2 = B2 + learning_rate * np.sum(output_delta, axis=0)
```

```
    B1 = B1 + learning_rate * np.sum(hidden_delta, axis=0)


# ---------------------------------------
# 5. Check Training Data
# ---------------------------------------

print("\nTraining Completed!")
print("\nChecking Training Data:")

for i in range(len(X)):

    hidden = sigmoid(np.dot(X[i:i+1], W1) + B1)
    result = sigmoid(np.dot(hidden, W2) + B2)

    predicted = 1 if result[0][0] >= 0.5 else 0

    actual_label = "Pass" if Y[i][0] == 1 else "Fail"
    predicted_label = "Pass" if predicted == 1 else "Fail"

    print(
        "Hours:", int(X[i][0] * 10),
        "Attendance:", int(X[i][1] * 100),
        "Actual:", actual_label,
        "Predicted:", predicted_label
    )


# ---------------------------------------
# 6. Test New Student
# ---------------------------------------

print("\n-----------------------------------")
print("TEST THE TRAINED EBPTA SYSTEM")
print("-----------------------------------")
```

```
hours = float(input("Enter Hours Studied: "))
attendance = float(input("Enter Attendance (%): "))

# Normalize input
new_student = np.array([[hours / 10, attendance / 100]])

# Forward pass for new student
hidden = sigmoid(np.dot(new_student, W1) + B1)
result = sigmoid(np.dot(hidden, W2) + B2)

predicted = 1 if result[0][0] >= 0.5 else 0

predicted_label = "Pass" if predicted == 1 else "Fail"

print("\nEBPTA Prediction:", predicted_label)
print("Output Value:", round(result[0][0], 4))


# ----------------------------------------
# 7. Verify Prediction
# ----------------------------------------

print("\nTo check whether the prediction is correct:")

actual = int(input("Enter Expected Target (0 = Fail, 1 = Pass): "))

if predicted == actual:
    print("Prediction is CORRECT")
else:
    print("Prediction is INCORRECT")

print("-----------------------------------")
```

Output: Input:
[[0, 0], [0, 1], [1, 0], [1, 1]]
Actual Output:
[0, 1, 1, 0]
Predicted Output:

Class: TE **Subject: AISC** **Sem-V**

[0 1 1 0]

Code:

https://colab.research.google.com/drive/1tLPhWlfld2Gk40d9Dt6EPh5cq1mKPL7m?usp=sharing

Conclusion:

The **EBPTA algorithm** was successfully implemented using Python to train a multilayer neural network for student performance prediction. The network learned by calculating prediction errors and adjusting its weights and biases through back propagation. After training, new student data could be entered to obtain a Pass/Fail prediction and compare it with the expected target.

Exercise 1:

*Add **Assignment Marks** as a third input.*

Use:

Hours	***Attendance***	***Assignment Marks***	***Result***
2	*50*	*40*	*Fail*
3	*55*	*45*	*Fail*
4	*60*	*50*	*Fail*
5	*70*	*65*	*Pass*
6	*75*	*70*	*Pass*
7	*85*	*80*	*Pass*

Exercise 2: Change the learning rate to 0.1

Class: TE **Subject: AISC** **Sem-V**